The Beginner's Practical Guide To Terraform Data Sources

Terraform Data Sources allow you to read existing cloud infrastructure information without creating or modifying those resources. Master this core concept to avoid hardcoding resource IDs and make your infrastructure code flexible and reusable across environments.

What you'll learn:

1. Fetch the Latest Machine Image dynamically
2. Connect to Existing VPC Networks
3. Read Current AWS Account Information
4. Reference Existing Security Groups

Aman Raj Singh
Cloud Operations Engineer @ iManage
linkedin.com/in/mramanrajsingh

1

TIP 1 OF 4

Fetch the Latest Machine Image dynamically

Hardcoding AMI IDs in Terraform breaks your configuration when images are deprecated or updated. Using the 'aws_ami' data source lets Terraform query the AWS API dynamically for the newest image matching your filters. You filter by name, virtualization type, and owner to always fetch the correct version automatically.

```
data "aws_ami" "ubuntu" {
  most_recent = true
  owners      = ["099720109477"] # Canonical

  filter {
    name   = "name"
    values = ["ubuntu/images/hvm-ssd/ubuntu-jammy-22.04-amd64-server-*"]
  }
}
```

Terraform Data Sources: Read Existing Cloud Resources in Code

Wednesday, November 04, 2026

Swipe to tip 2 >> linkedin.com/in/mramanrajsingh

2

TIP 2 OF 4

Connect to Existing VPC Networks

You rarely create a brand new VPC for every small project; most applications run inside an existing company network. The 'aws_vpc' data source allows your Terraform code to query existing VPC details like CIDR blocks or VPC IDs using tags or default settings.

```
data "aws_vpc" "selected" {
  tags = {
    Name = "production-vpc"
  }
}

# Accessing VPC ID elsewhere:
# aws_subnet.example.vpc_id = data.aws_vpc.selected.id
```

Terraform Data Sources: Read Existing Cloud Resources in Code

Wednesday, November 04, 2026

Swipe to tip 3 >> linkedin.com/in/mramanrajsingh

3

TIP 3 OF 4

Read Current AWS Account Information

To make Terraform code portable across multiple AWS accounts, avoid typing your 12-digit AWS Account ID manually. The 'aws_caller_identity' data source returns information about the account, ARN, and user ID running the Terraform apply command.

```
data "aws_caller_identity" "current" {}

output "account_id" {
  value = data.aws_caller_identity.current.account_id
}
```

Swipe to tip 4 >> linkedin.com/in/mramanrajsingh

4

TIP 4 OF 4

Reference Existing Security Groups

Centralized IT teams often pre-create firewall security groups for web or database traffic. By querying these groups with 'aws_security_group', you can attach existing network security rules to your newly launched virtual instances without modifying original rules.

```
data "aws_security_group" "web" {
  name = "allow-web-traffic"
}

# Reference ID in instance:
# vpc_security_group_ids = [data.aws_security_group.web.id]
```

See Key Takeaways on next page >> linkedin.com/in/mramanrajsingh

Key Takeaways

+ Use tags to make querying existing resources simple and accurate
+ Always set 'most_recent = true' when searching for dynamic images like AMIs
+ Store resource IDs as outputs to easily verify fetched data using 'terraform output'
+ Keep data source queries specific to avoid returning multiple resources unexpectedly
+ Treat data sources as read-only references; Terraform will never delete them on destroy

Watch Out For

! Expecting 'terraform destroy' to delete resources referenced in a data block
! Writing filters that return multiple resources when Terraform expects exactly one result
! Hardcoding IDs inside data source filters instead of using dynamic tags or names

Follow for daily Cloud & DevOps guides!

linkedin.com/in/mramanrajsingh